

Software Engineering

Chapter 4: System Modeling & Unified Architecture

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Chapter 4: Roadmap & Core Curriculum

Part 1: Foundations & Functional Models

- **4.1 Foundations of Modeling & UML:** What is modeling, 4 perspectives, 5 essential diagrams, method wars, and Three Amigos name cards.
- **4.2 Context & Process Models:** System boundaries, external partners, and activity workflows.
- **4.3 Functional & Use Case Models:** Jacobson's legacy, include vs. extend, structured specifications.

Part 2: Structural, Dynamic & AI Modeling

- **4.4 Sequence Diagrams & BCE:** Message passing, lifelines, activation, Boundary-Control-Entity pattern.
- **4.5 Structural & Class Models:** Domain classes, visibility, multiplicity, composition vs. aggregation.
- **4.6 Behavioral & State Machines:** Order lifecycle FSM, event triggers, guards, actions.
- **4.7 AI-Assisted Modeling:** Text-to-UML copilot, human-in-the-loop validation, conceptual recap.

4.1 Foundations of System Modeling & UML

"A language that doesn't affect the way you think about programming is not worth knowing."
— Alan Perlis

”

What is System Modeling?

"System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system."

— Ian Sommerville, Software Engineering (10th ed.)

”

- **Essential Principles of Modeling:**

- **Abstraction:** Hides non-essential implementation details to highlight critical architectural structures, data flows, and dependencies.
- **Multiple Perspectives:** No single diagram can explain a complex software system. Different stakeholders require different views.
- **Communication & Verification:** Serves as a universal visual grammar between product managers, system architects, software developers, and QA engineers.
- **Blueprint for Implementation:** Provides the formal structural and behavioral specification from which executable code and database schemas are developed.

4 Core System Modeling Perspectives

- **1. External Perspective:**
 - Models the environment, external partners, and operational context of the system.
 - Defines the strict system perimeter: what is built internally vs. what is delegated to third parties.
- **2. Interaction Perspective:**
 - Models dynamic communications between external actors and the system, or message exchanges between internal collaborating objects.
- **3. Structural Perspective:**
 - Models the static architecture of system data, object classes, attributes, methods, and relationships independent of runtime execution order.
- **4. Behavioral Perspective:**
 - Models the dynamic execution behavior, sequential business workflows, and reactive discrete state transitions in response to external events.

5 Essential UML Diagram Types in Modern Practice

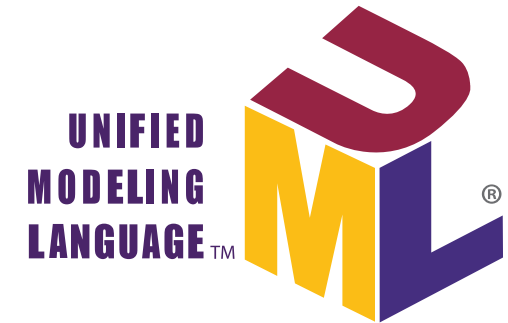
Diagram Type	Perspective	Dynamic / Static	Primary Engineering Role
1. Context Diagram	External	Static Boundary	Defines system boundaries and external partner systems
2. Activity Diagram	Behavioral	Dynamic Flow	Visualizes sequential business workflows and concurrent logic
3. Use Case Diagram	Interaction	Static Contract	Specifies actor goals and functional scope boundaries
4. Sequence Diagram	Interaction	Dynamic Time	Traces chronological message passing across runtime lifelines
5. Class Diagram	Structural	Static Backbone	Specifies domain entities, relationships, attributes, and methods
<i>(Bonus) State Diagram</i>	Behavioral	Dynamic Reactive	Models discrete states and event-driven reactive lifecycles

The Need for Standardization: The 1990s "Method Wars"

- **The Rise of Object-Oriented Programming (Late 1980s – Early 1990s):**
 - The software industry transitioned from procedural code (C, Pascal) to object-oriented paradigms (C++, Smalltalk).
 - Software engineers urgently needed visual notations to represent classes, objects, and relationships.
- **The "Method Wars" Era:**
 - Over **50 competing OO modeling notations** flooded the commercial market.
 - Engineers argued fiercely over whether classes should be clouds, rectangles, or ovals; whether inheritance should be an open triangle, filled arrow, or dashed line.
 - **Severe Industry Fragmentation:** Companies could not exchange models, CASE tools were incompatible, and developers had to relearn notation whenever they switched jobs.

Unification & Standardization: From Rational to OMG

- **1994 – Unification Begins at Rational Software:**
 - Jim Rumbaugh joined Grady Booch at Rational to merge the Booch Method and OMT into the "Unified Method" (v0.8).
- **1995 – The Three Amigos Assemble:**
 - Ivar Jacobson joined Rational, bringing his revolutionary **Use Case** methodology (OOSE).
- **1997 – OMG International Standardization:**
 - Submitted to the **Object Management Group (OMG)**; unanimously adopted as **UML 1.1** in November 1997.
- **2005 – UML 2.0 Major Architecture Overhaul:**
 - Expanded from 9 to 13 (and later 14) diagram types with formal execution metamodels.



Unified Modeling Language
Object Management Group (OMG)

Pioneers of UML: Grady Booch

- **Role & Distinctions:**

- Chief Scientist, Rational Software; IBM Fellow; ACM Fellow.

- **Pioneered Methodology:**

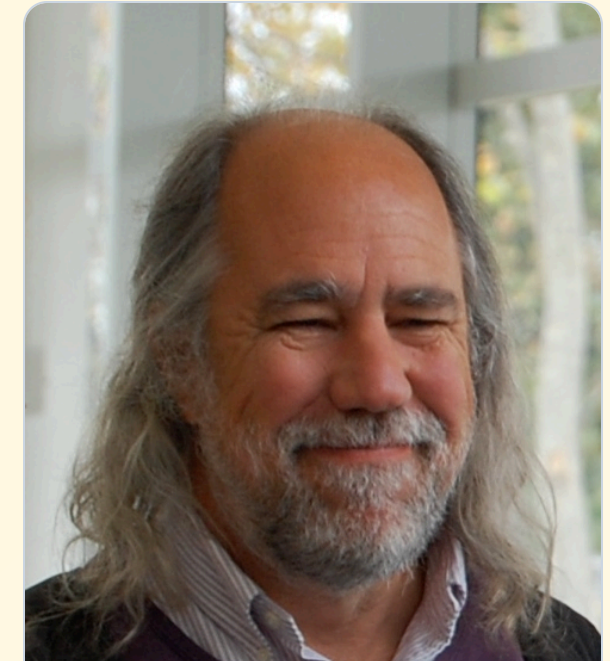
- **The Booch Method** and seminal text: *Object-Oriented Analysis and Design with Applications*.

- **Core Contribution to UML:**

- Focused heavily on **concrete software design**, module decomposition, class abstractions, and architectural patterns.
- Championed visual expressiveness for implementation-level object structures and code mapping.

- **Famous Architectural Maxim:**

"Clean code always looks like it was written by someone who cares."



Grady Booch

Rational Software / IBM Fellow

Pioneers of UML: James Rumbaugh

- **Role & Distinctions:**

- Lead Researcher, General Electric (GE) Global R&D; Rational Software; IBM.

- **Pioneered Methodology:**

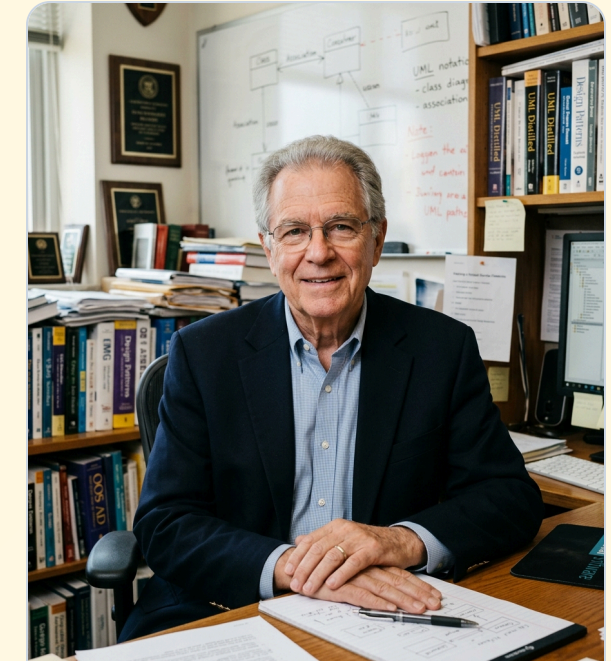
- **Object Modeling Technique (OMT)** and book: *Object-Oriented Modeling and Design*.

- **Core Contribution to UML:**

- Emphasized rigorous **domain analysis**, semantic data modeling, and entity-relationship mapping.
- Pioneered the synthesis of object structure with David Harel's **Statecharts** to model dynamic reactive systems.

- **Famous Architectural Maxim:**

"You cannot build great software without understanding domain truth."



James Rumbaugh

GE Research / Rational Software

Pioneers of UML: Ivar Jacobson

- **Role & Distinctions:**

- Lead Architect, Ericsson; Founder, Objectory AB; Rational Software; SEMAT Pioneer.

- **Pioneered Methodology:**

- **Object-Oriented Software Engineering (OOSE).**

- **Core Contribution to UML:**

- **Inventor of Use Cases (1986):** Revolutionized requirements by anchoring system architecture to measurable user goals.
- Introduced the **Boundary–Control–Entity (BCE)** robustness analysis pattern and component-based architecture.

- **Famous Architectural Maxim:**

"A system that has no users has no reason to exist. Model user goals first."



Ivar Jacobson

Ericsson / Objectory / Rational

The Three Amigos: The Trinity of Software Modeling

Ivar Jacobson

The "Why" (Goal / User)

- **Use Case Driven:** What goals do external actors achieve?
- **BCE Robustness:** Separating UI boundaries from data entities.
- **Component Contracts:** Reusable subsystem architectures.

James Rumbaugh

The "What" (Domain Data)

- **Domain Analysis:** What real-world concepts exist?
- **Class & Data Schemas:** Structural entity relationships.
- **State Machines:** Reactive event-driven lifecycles.

Grady Booch

The "How" (Design & Code)

- **Object-Oriented Design:** How do classes collaborate?
- **Macro Architecture:** Layered module decomposition.
- **Implementation Map:** Direct code translation into OOP.

📌 **Architectural Synthesis:** *Jacobson captures **Why** the system exists; Rumbaugh models **What** domain concepts exist; Booch designs **How** components execute in code. „*

Concept Check Question 1

Which of the "Three Amigos" was specifically renowned for inventing **Use Cases (1986)** to anchor software architecture to tangible user goals?

- **A.** Grady Booch
- **B.** James Rumbaugh
- **C.** Ivar Jacobson
- **D.** Martin Fowler



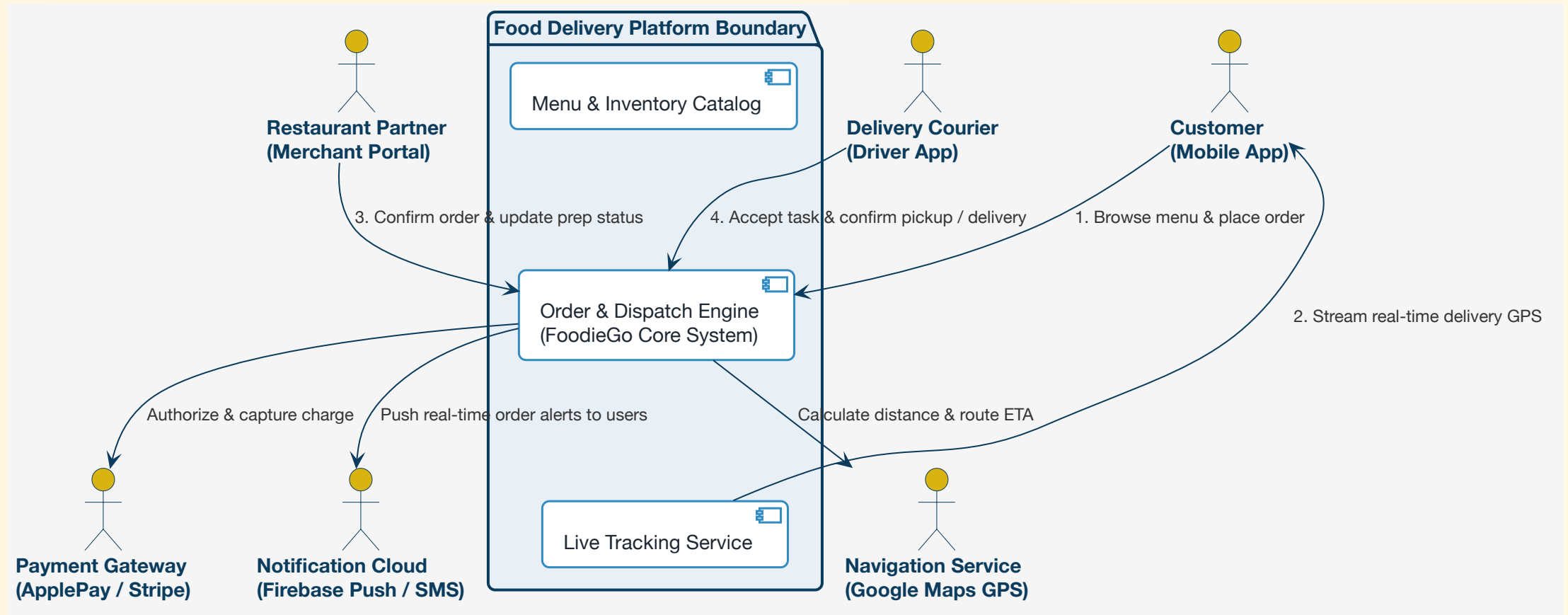
4.2 Context & Process Models

"Architecture is the decisions that you wish you could get right early in a project."

— *Ralph Johnson*

”

Food Delivery Platform: System Context Model



Explaining the Context Model & Architectural Boundaries

- **The Primary Operational Boundary:**

- Clearly demarcates what is **internal** (our software platform: Order Engine, Catalog, Tracking) versus what is **external** (third-party systems and human actors).
- Eliminates "scope creep" by defining external dependencies early in the architectural lifecycle.

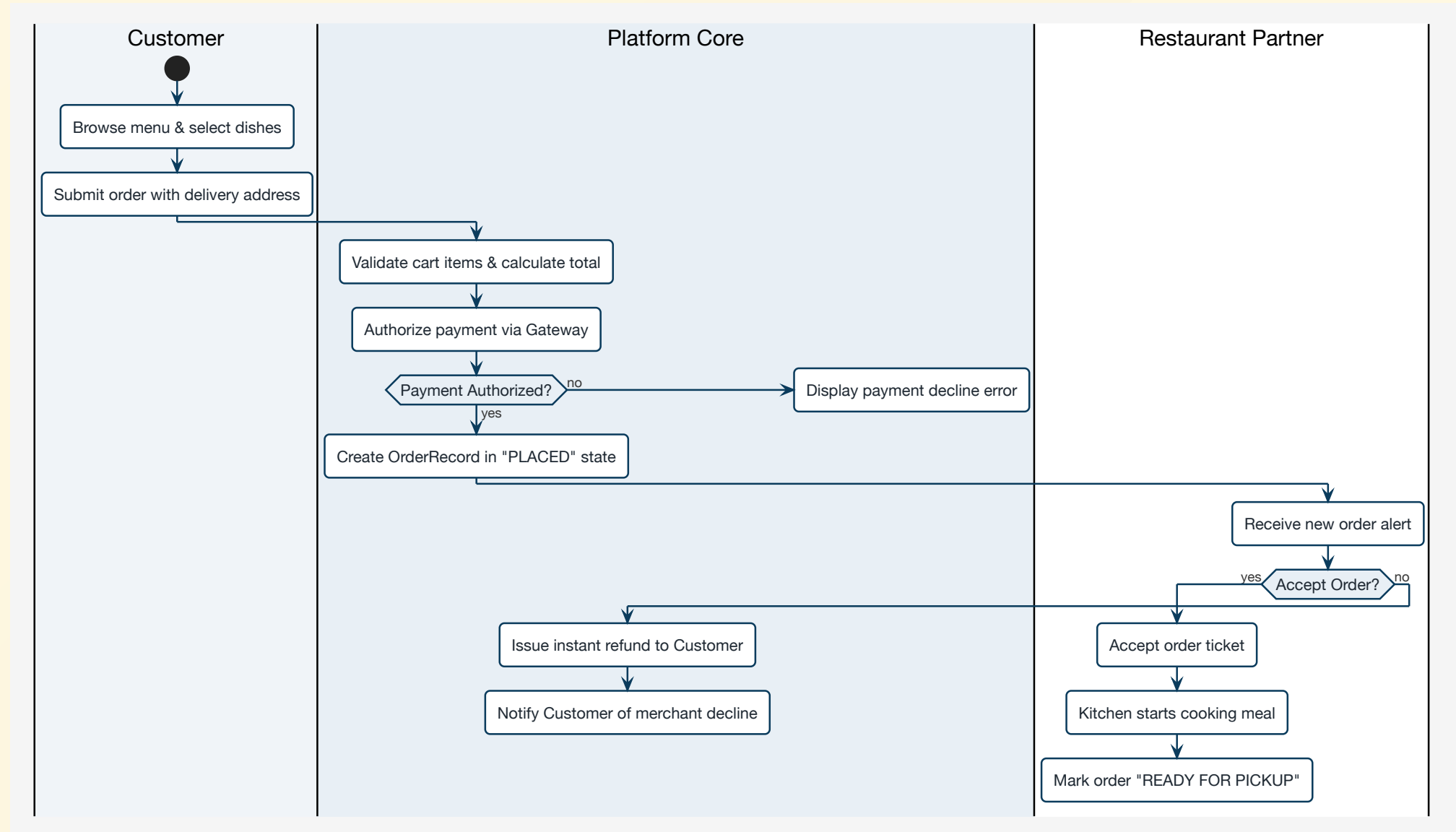
- **Human Stakeholder Interfaces:**

- **Customer:** Interacts via native iOS/Android mobile apps to query menus and submit orders.
- **Restaurant Partner:** Interacts via merchant web/tablet portals to accept orders and manage food availability.
- **Delivery Courier:** Interacts via courier mobile apps to receive dispatch missions and stream GPS coordinates.

- **External Third-Party Service Dependencies:**

- **Payment Gateway (Stripe/ApplePay):** Handles PCI-compliant financial transactions without storing raw card data internally.
- **Navigation Service (Google Maps API):** Computes live routing distances, courier ETAs, and road geometry.
- **Notification Cloud (Firebase / Twilio):** Dispatches asynchronous push alerts and SMS verification tokens.

Food Delivery Fulfillment: Order Placement & Kitchen Prep



Explaining Stage 1 Activity Flow & Notations

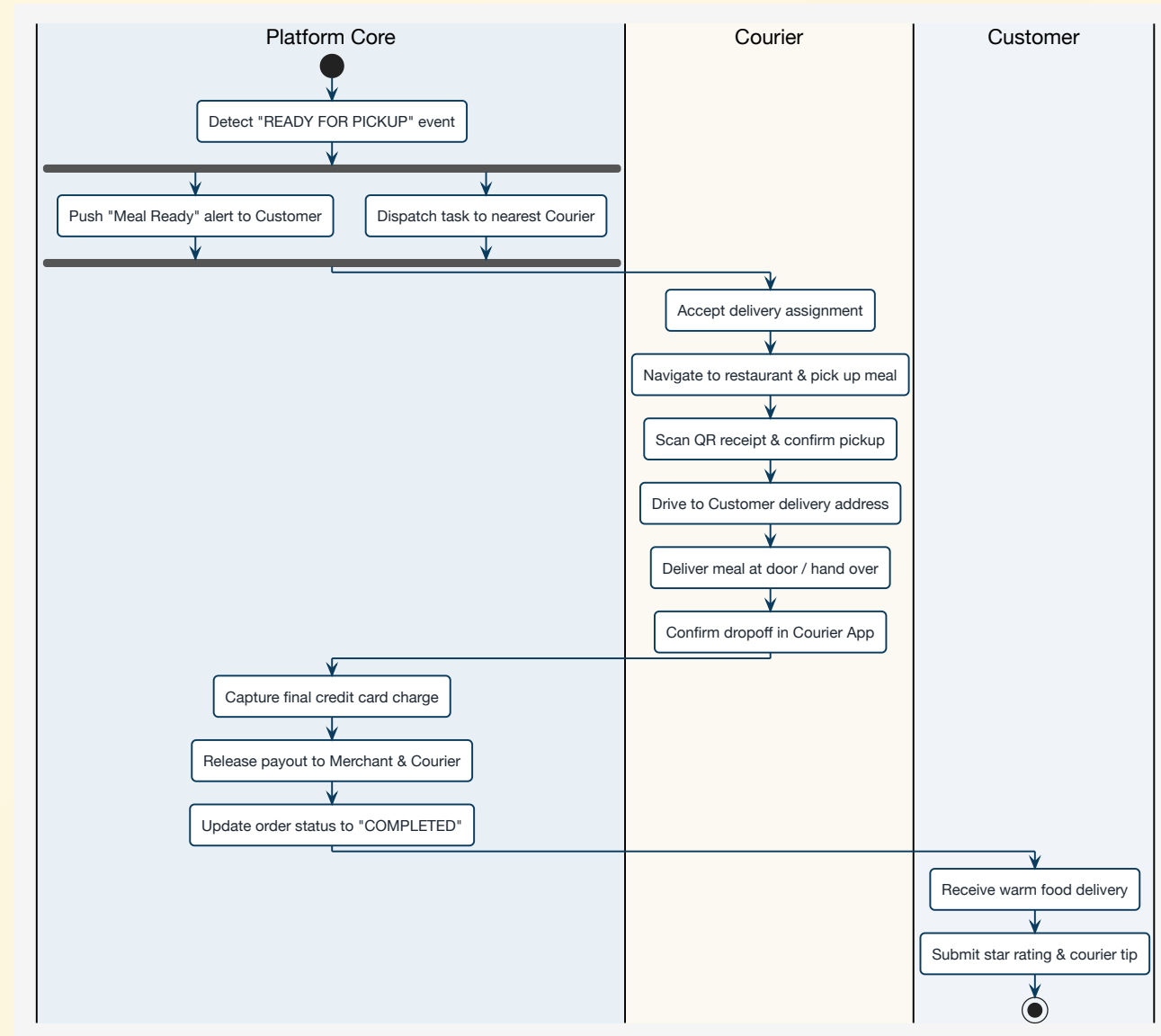
- **Core Notation Elements in Activity Modeling:**

- **Rounded Rectangles:** Operational action states (e.g., `Browse menu & select dishes`, `Kitchen starts cooking meal`).
- **Swimlanes (Partitions):** Assign direct organizational ownership to Customer, Platform Core, and Restaurant Partner.
- **Decision Diamonds:** Branching logic based on boolean guard conditions (`[Payment Authorized?]`, `[Accept Order?]`).
- **Detach Node / Terminal Exit:** Represents an abnormal branch termination (e.g., payment decline or restaurant rejection).

- **Engineering Rationale for Decoupling:**

- Notice that Customer interaction ends at order submission; subsequent steps are managed asynchronously by the Platform and Merchant.
- If a restaurant rejects an order, the Platform Core autonomously triggers an instant automated refund.

Food Delivery Fulfillment: Courier Dispatch & Delivery Handover



Explaining Stage 2 Activity Flow: Concurrency & Fork/Join

- **Fork and Join Synchronization Bars (Thick Horizontal Bars):**
 - **Fork Bar (Concurrency Generator):** Splits a single control thread into **parallel concurrent activities**.
 - *Example:* The platform concurrently alerts the customer AND dispatches the courier. Neither branch blocks the other!
 - **Join Bar (Synchronization Barrier):** Recombines concurrent threads; execution proceeds only after *all* incoming parallel branches complete.
- **Real-World Fulfillment Mechanics:**
 - **Physical-to-Digital Handshake:** The courier scans a QR code receipt to atomically confirm meal pickup.
 - **Financial Settlement Trigger:** Final credit card charges and merchant/courier payouts are captured *only upon verified physical delivery*.
 - **Customer Feedback Loop:** Completing dropoff immediately triggers the rating and tipping prompt.

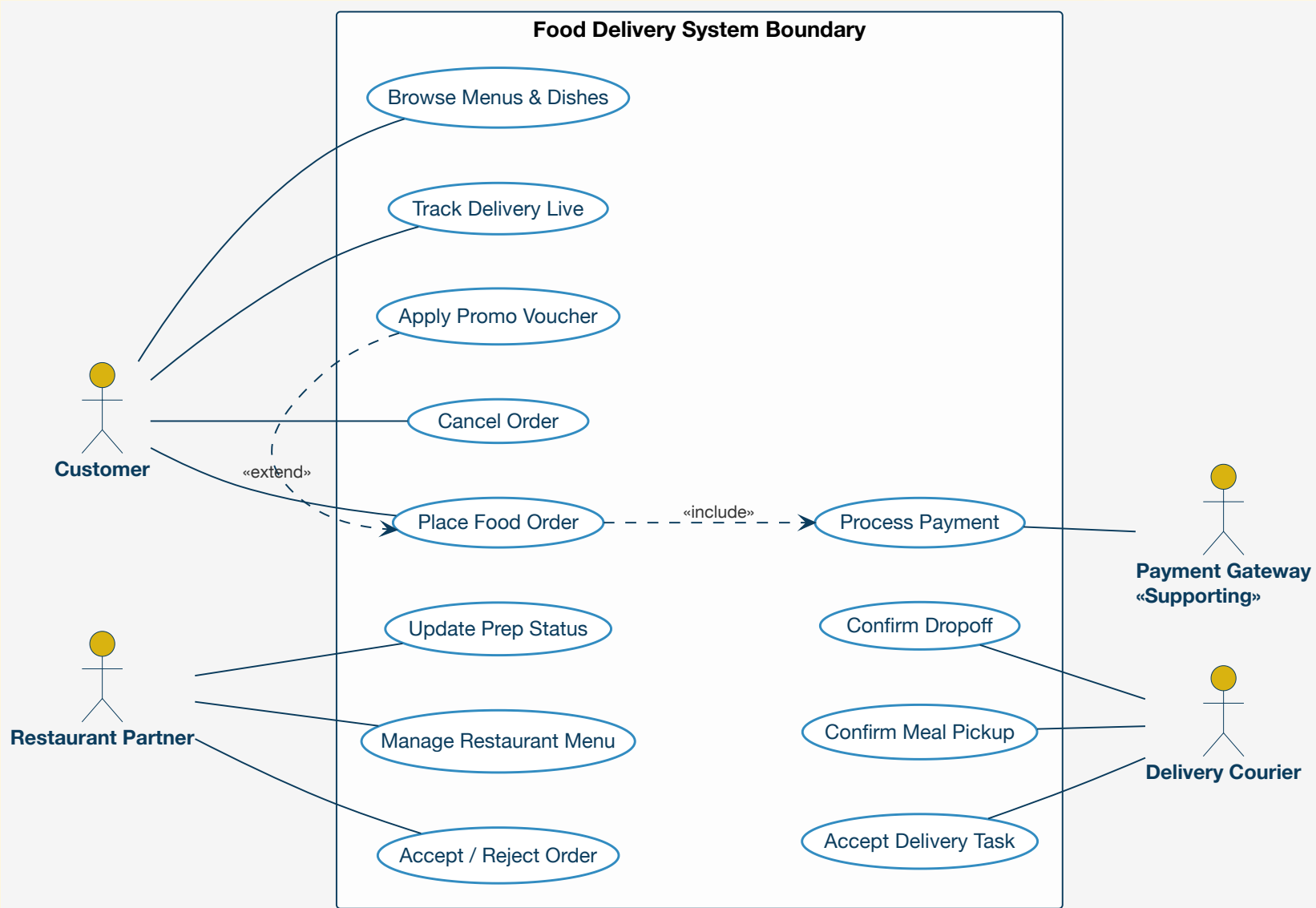
4.3 Functional & Use Case Models

"A use case is a contract for behavior between the system and its actors to achieve a measurable business goal."

— Alistair Cockburn

”

Food Delivery Platform: Use Case Diagram



Explaining the Use Case Model & Actor Roles

- **Primary vs. Supporting Actors:**

- **Primary Actors (Customer, Restaurant, Courier):** Human users who actively initiate use cases to achieve a personal business goal (e.g., eat food, earn revenue, deliver meals).
- **Supporting / Secondary Actors (Payment Gateway):** External services invoked by the system to assist in fulfilling a use case (e.g., authenticating credit cards).

- **System Boundary Demarcation:**

- All use case ovals reside **inside** the boundary; actors reside **outside**.
- A line connecting an actor to an oval indicates that the actor participates in that functional interaction.

- **Granularity Rule of Thumb:**

- A use case must represent a **complete, value-delivering transaction**.
- *Bad:* "Enter Password", "Click Submit Button" (These are trivial UI actions, not use cases!).
- *Good:* "Place Food Order", "Manage Restaurant Menu" (Delivers measurable business value).

Notation Breakdown: <<include>> vs. <<extend>>

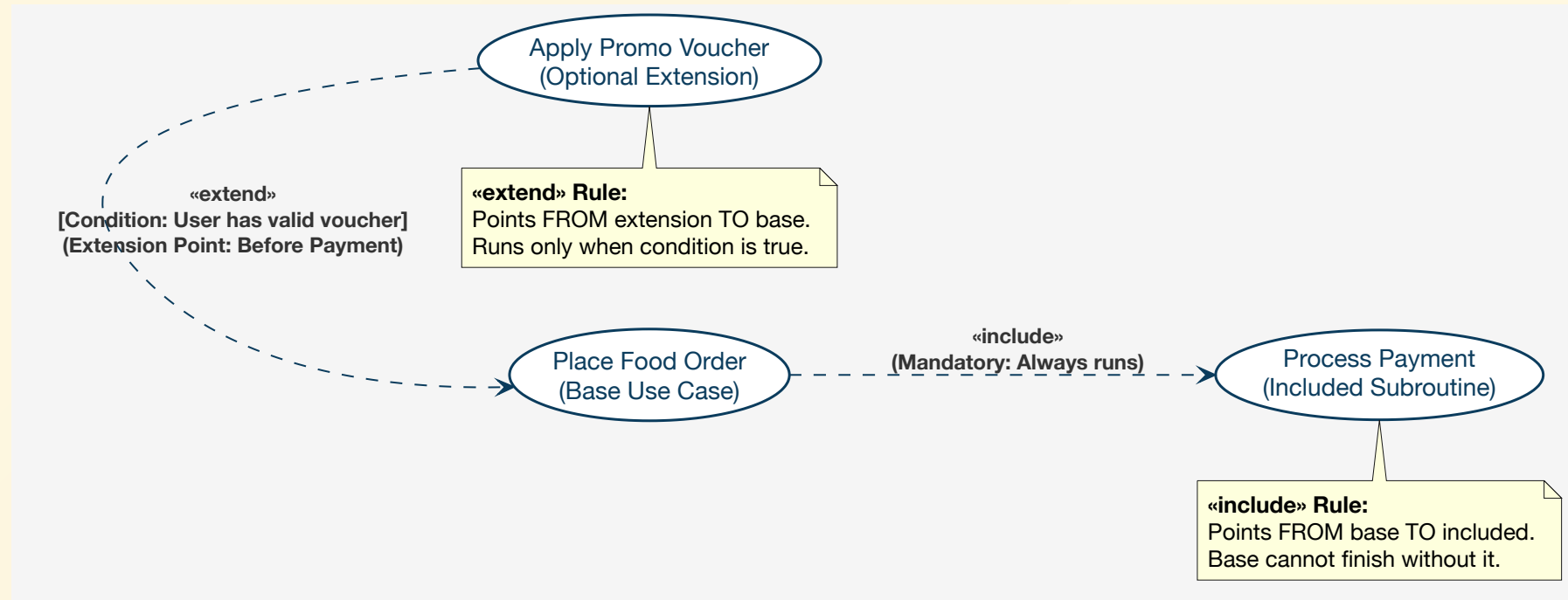
<<include>> (Mandatory)

- **Semantics:** The base use case **cannot complete** without executing the included subroutine.
- **Arrow Direction:** Points **from base toward included** (`Base ..> Included`).
- **Engineering Purpose:** Factors out common mandatory logic into reusable sub-cases (e.g., multiple order flows all require *Process Payment*).

<<extend>> (Conditional)

- **Semantics:** Optional behavior is inserted into the base flow **only under specific trigger conditions**.
- **Arrow Direction:** Points **from extension toward base** (`Extension ..> Base`).
- **Engineering Purpose:** Isolates optional edge cases (e.g., promo codes, contactless dropoff) without bloating the base happy path.

Include & Extend: Visual Syntax & Arrow Mechanics



Structured Use Case Specification: Place Food Order

Specification Field	Technical Description & Contract
Use Case ID & Name	UC-01: Place Food Order
Primary Actor	Customer (Registered FoodieGo User)
Preconditions	Customer is authenticated; cart contains ≥ 1 available dish from an open merchant.
Postconditions	Order recorded in <code>PAID</code> state; kitchen alerted; courier dispatch queued; receipt emailed.
Main Success Scenario	1. Customer initiates checkout from cart review screen. 2. System validates real-time dish availability and calculates tax, tip, and delivery fee. 3. System executes <code><<include>></code> UC-02: Process Payment via Payment Gateway. 4. System creates persistent <code>OrderRecord</code> with unique <code>orderId</code>. 5. System transmits order ticket to Restaurant Partner dashboard. 6. System initializes live GPS tracking session and displays estimated delivery time.
Extensions (Alternate)	3a. Payment Authorization Denied: 3a1. System notifies Customer of card decline; order remains in <code>DRAFT</code> state. 4a. <code><<extend>></code> Apply Promo Voucher: 4a1. Customer submits valid voucher code; system deducts discount before Step 3.

Concept Check Question 2

In our Food Delivery Use Case Model, why does `Apply Promo Voucher` point to `Place Food Order` with `<<extend>>`, while `Place Food Order` points to `Process Payment` with `<<include>>`?

- **A.** Voucher application is mandatory for all orders; payment is optional.
- **B.** Vouchers are conditional optional behavior; payment is mandatory shared execution.
- **C.** Vouchers are executed by supporting actors; payment is executed by primary actors.
- **D.** Vouchers represent class inheritance; payment represents object composition.

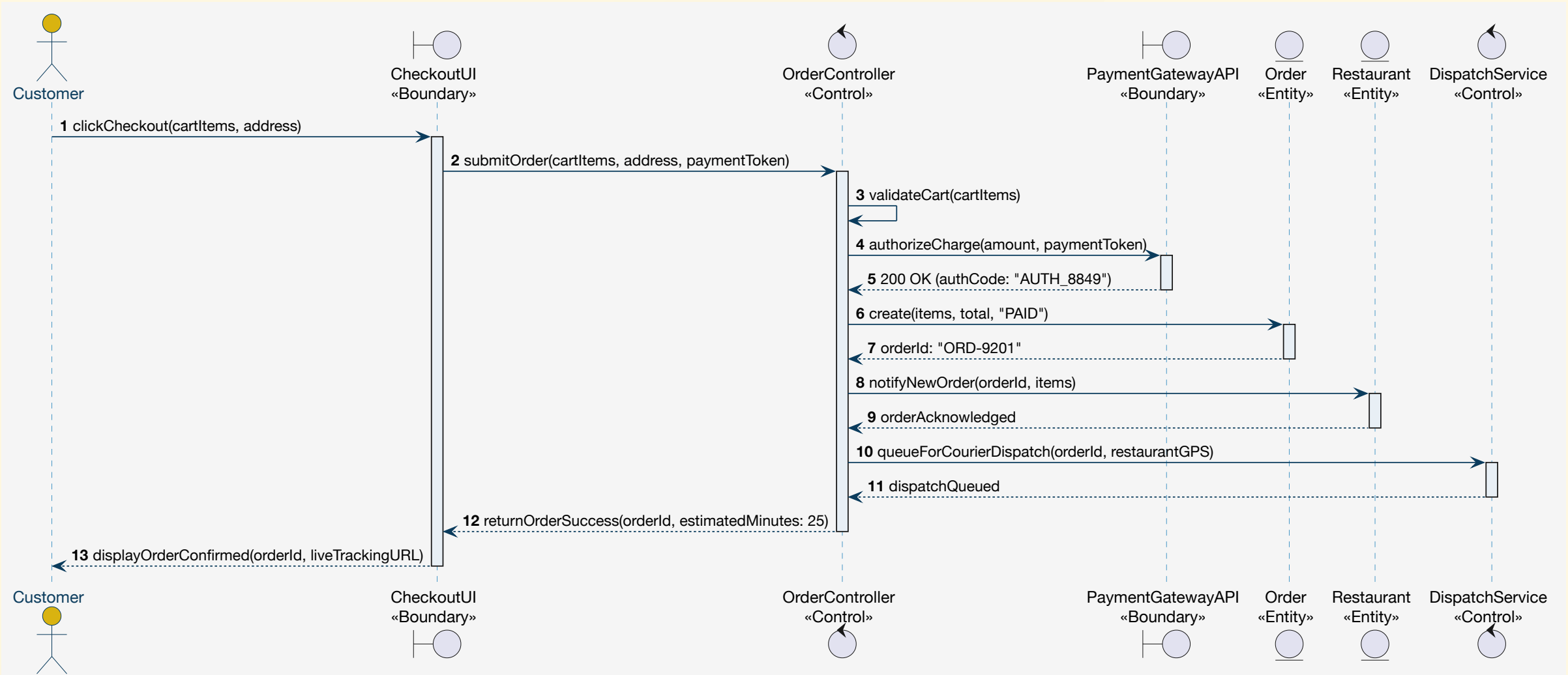


4.4 Sequence Diagrams & BCE

"Interaction modeling shows how objects collaborate over time to fulfill the promise of a use case."

”

Order Placement & Payment Flow: Sequence Diagram



Explaining the Sequence Flow & BCE Architecture

- **The Boundary–Control–Entity (BCE) Architectural Pattern:**
 - **Boundary Objects** (`<<Boundary>>`): Handle communication with actors and external APIs (e.g., `CheckoutUI`, `PaymentGatewayAPI`).
 - **Control Objects** (`<<Control>>`): Orchestrate transaction workflows and business rules (e.g., `OrderController`, `DispatchService`).
 - **Entity Objects** (`<<Entity>>`): Encapsulate persistent domain state and business data (e.g., `Order`, `Restaurant`).
- **The Non-Negotiable Engineering Rule of BCE:**
 - External actors and UI boundaries must **never interact directly with Entity data objects!**
 - Interactions must flow strictly: **Actor** → **Boundary** → **Control** → **Entity**.
 - *Why?* Decouples the UI from the database schema. If the database schema changes, UI code remains completely unaffected.

Sequence Diagram Notations & Semantics Guide

Visual Symbol	Notation Element	Precise Technical Semantics
Vertical Dashed Line	Lifeline	Represents the existence of an active object instance in memory over time.
Narrow Vertical Box	Activation Bar	Duration during which the object instance is actively executing code on CPU.
Solid Arrow (Filled Head)	Synchronous Call	Blocking call; caller halts execution waiting for return response.
Solid Arrow (Open Head)	Asynchronous Call	Non-blocking message; caller fires message and continues execution immediately.
Dashed Arrow (Open Head)	Return Message	Explicitly returns computational results or data back to the original caller.
Self-Loop Arrow	Self-Invocation	An object instance executing its own internal private method (e.g., <code>validateCart()</code>).

 **Geometric Golden Rule:** Time advances strictly **downwards** along the vertical axis. Message arrows must never point upwards!

Concept Check Question 3

In a UML sequence diagram designed with the **Boundary–Control–Entity (BCE)** architecture, which object should directly receive the user's checkout submission from `CheckoutUI`?

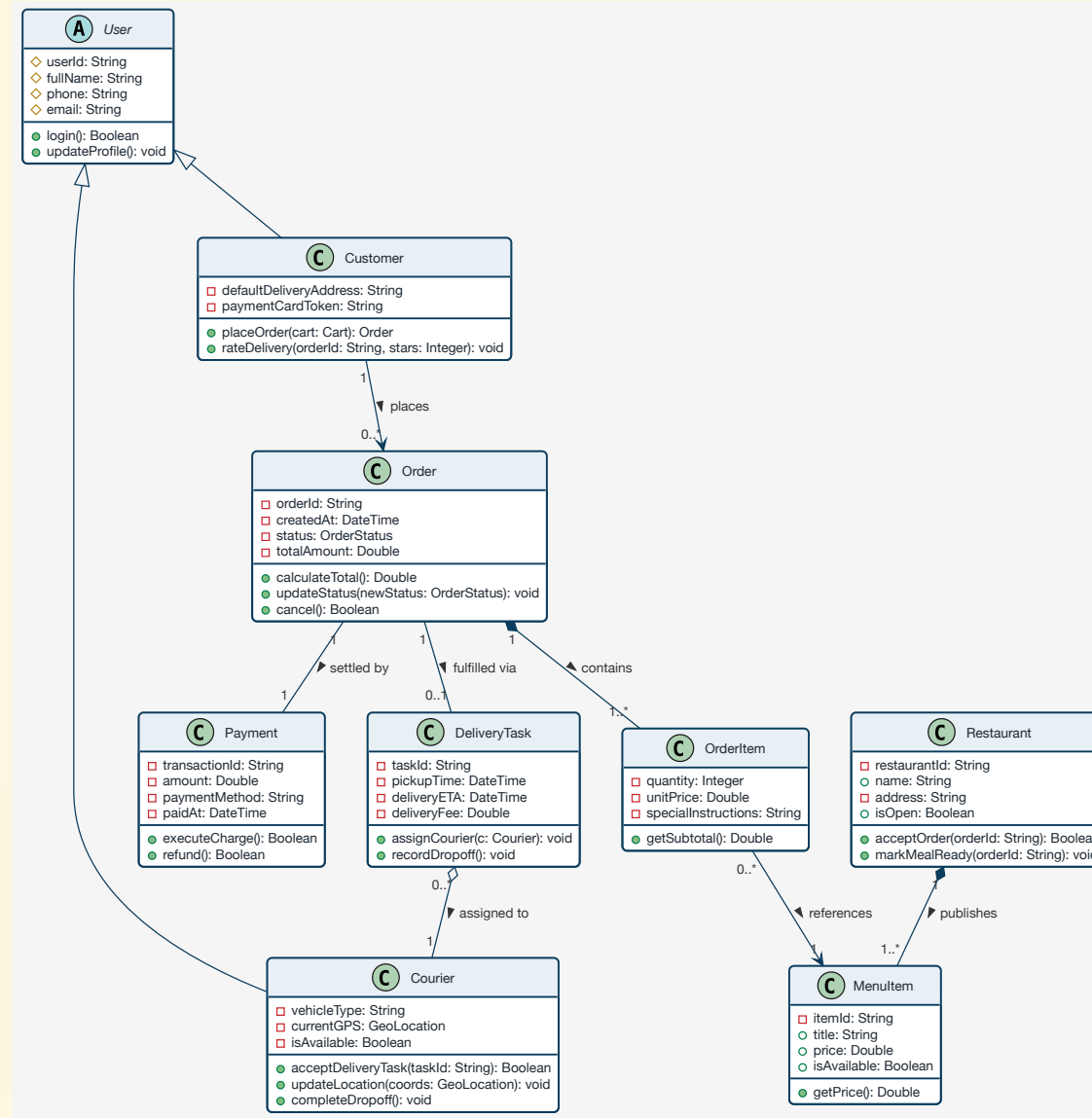
- **A.** The `Order` entity object to immediately save data to the database.
- **B.** The `PaymentGatewayAPI` boundary object to process payment first.
- **C.** The `OrderController` control object to orchestrate business validation.
- **D.** The `Restaurant` entity object to confirm kitchen capacity.



4.5 Structural & Class Models

"Classes are the static building blocks; objects are the living runtime instances." „

Food Delivery Domain: Class Diagram Blueprint



Explaining the Domain Class Structure & Object Lifecycles

- **Generalization Hierarchy (Inheritance):**

- `User` is an abstract superclass defining shared attributes (`userId`, `phone`, `email`) and methods (`login()`).
- `Customer` and `Courier` specialize `User`, inheriting core identity while adding role-specific fields (e.g., delivery address vs. vehicle type and live GPS).

- **Composition (◆ Solid Diamond) – Strong Whole-Part Ownership:**

- `Order "1" *-- "1..*" OrderItem`: An `OrderItem` (e.g., 2 Spicy Burgers) has no independent existence outside its parent `Order`. If an order is deleted, all its `OrderItem` instances are cascade-deleted!
- `Restaurant "1" *-- "1..*" MenuItem`: Menu dishes belong strictly to their publishing restaurant.

- **Aggregation (◇ Hollow Diamond) – Weak Whole-Part Relationship:**

- `DeliveryTask "0..*" o-- "1" Courier`: A delivery task is assigned to a courier, but the `Courier` **exists independently**. If the task is completed or cancelled, the courier does not vanish!

- **Association vs. Catalog Independence:**

- `OrderItem` references `MenuItem` with `0..* --> 1`. An order item captures the historical purchase price at checkout time, decoupling it from future restaurant menu price

Class Diagram Notations: Compartments, Visibility & Multiplicity

- **The Standard Three-Compartment Class Box:**

- **Top Compartment:** Class Name in `PascalCase` (italics denote an `abstract` class).
- **Middle Compartment:** Typed Attributes: `[visibility] name: Type [= defaultValue]`.
- **Bottom Compartment:** Operations: `[visibility] name(parameter: Type): ReturnType`.

- **Visibility Modifiers (Encapsulation Grammar):**

- **+** **Public:** Accessible by any class in the codebase.
- **-** **Private:** Strictly encapsulated within this class declaration.
- **#** **Protected:** Accessible within this class and derived subclasses.
- **~** **Package:** Accessible within the same module/namespace.

- **Association Multiplicities:**

- **1**: Exactly one instance required.
- **0..1**: Optional; zero or one instance.
- **0..*** (or *****): Zero or many instances.
- **1..***: At least one instance required (unbounded upper limit).

Concept Check Question 4

In our Food Delivery Class Diagram, why is the relationship between `Order` and `OrderItem` modeled as **Composition** (◈), whereas `DeliveryTask` and `Courier` is modeled as **Aggregation** (◊)?

- **A.** An `OrderItem` can exist independently, but a `Courier` cannot exist without a `DeliveryTask`.
- **B.** An `OrderItem` is destroyed with its parent `Order`, whereas a `Courier` maintains an independent lifecycle.
- **C.** Composition represents inheritance between classes, while aggregation represents method invocation.
- **D.** Composition requires zero-to-one multiplicity, while aggregation requires one-to-many multiplicity.

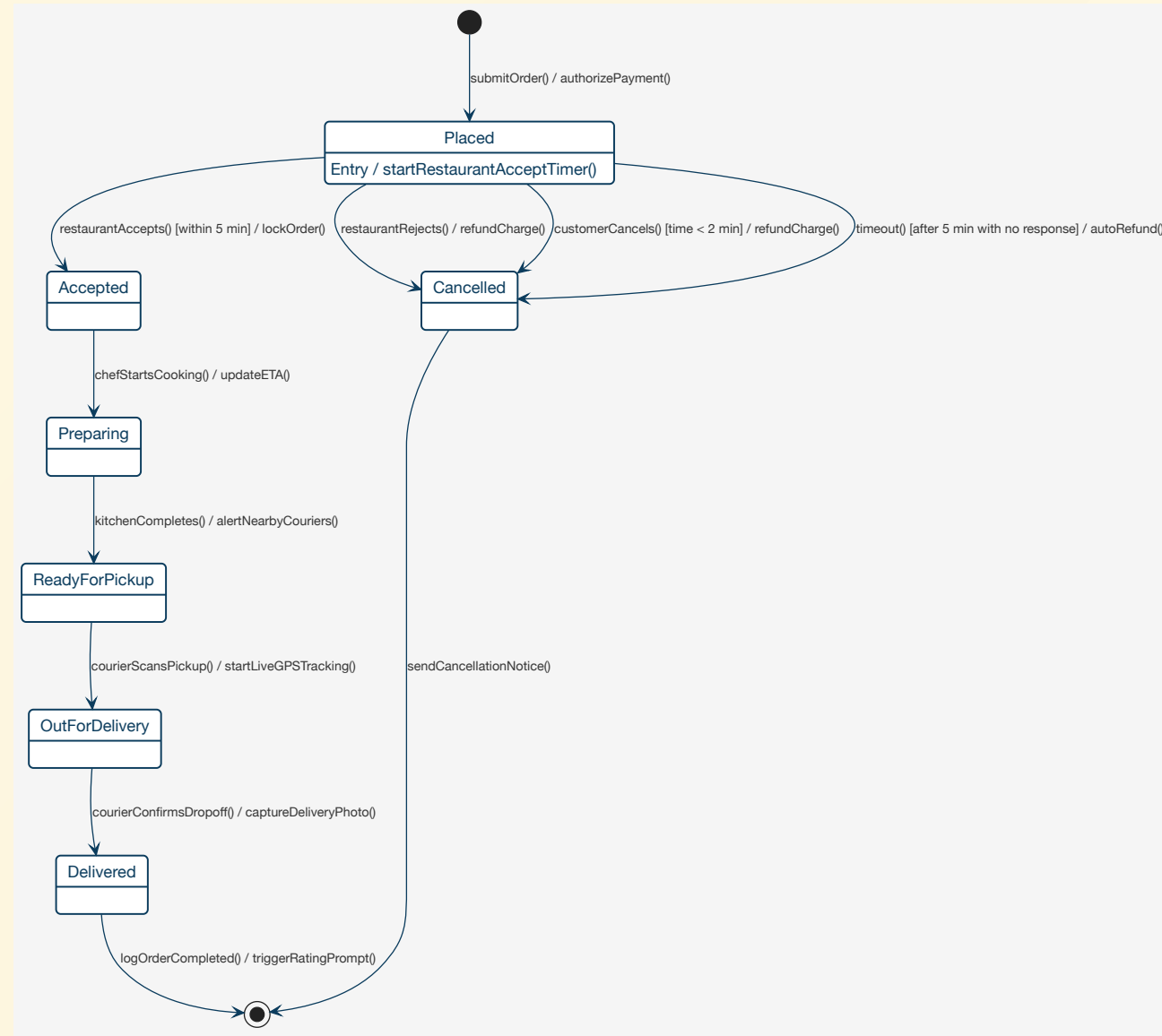


4.6 Behavioral & State Machine Models

"A system in dynamic execution is defined by the states it occupies and the events that trigger transitions."

”

Order Lifecycle: UML State Machine Diagram



Explaining the Statechart & Transition Mechanics

- **Finite State Machine (FSM) Principles:**

- At any single point in runtime execution, an `Order` instance resides in **exactly one** discrete state.
- The order's response to an incoming event depends entirely on its **current active state**.

- **The Formal Transition Label Grammar:**

Trigger Event [Guard Condition] / Action Effect

- **Trigger Event:** The stimulus that initiates the transition (e.g., `chefStartsCooking()`, `courierScansPickup()`).
- **Guard Condition (`[...]`):** A boolean condition that must evaluate to `true` for the transition to fire (e.g., `[within 5 min]`, `[time < 2 min]`).
- **Action Effect (`/ ...`):** An atomic computational operation executed during the transition (e.g., `/ refundCharge()`, `/ startLiveGPSTracking()`).

- **State Entry Actions:**

- An internal action executed automatically upon entering a state (e.g., `Placed: Entry / startRestaurantAcceptTimer()`).

Concept Check Question 5

In our Order Lifecycle State Machine, what does the label `customerCancels() [time < 2 min] / refundCharge()` specify?

- **A.** `customerCancels()` is the guard; `[time < 2 min]` is the event; `refundCharge()` is the state.
- **B.** `customerCancels()` is the event trigger; `[time < 2 min]` is the boolean guard; `refundCharge()` is the action.
- **C.** `customerCancels()` is the class; `[time < 2 min]` is the method; `refundCharge()` is the return type.
- **D.** `customerCancels()` is the primary actor; `[time < 2 min]` is the timeout; `refundCharge()` is the lifeline.



4.7 AI-Assisted System Modeling

"AI is a visual copilot that eliminates diagramming friction, but human architects must ensure semantic correctness."

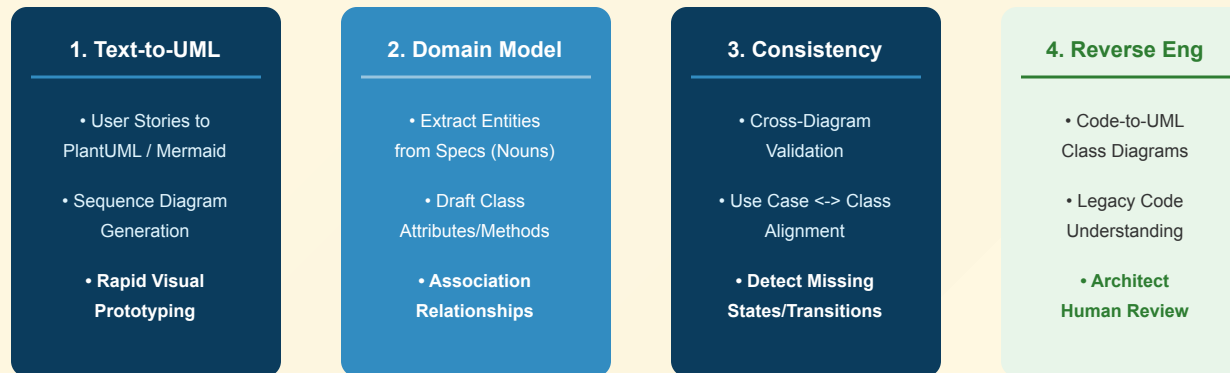
”

AI in System Modeling: The Visual Copilot

- **The Text-to-Diagram Revolution:**

- Large Language Models convert unstructured software requirements directly into **declarative diagram markup** (PlantUML, Mermaid.js, Graphviz).
- Bridges natural language user stories and formal graphical architecture models in seconds.

How AI Assists System Modeling (UML & Diagramming Copilot)



- **Core Efficiency Leap:** Eliminates tedious manual formatting, allowing engineers to focus on architectural reasoning rather than visual layout.

4 Core AI Applications in System Modeling

- **1. Text-to-UML Generation:**
 - Automatically generates Sequence, Class, and Activity diagrams directly from agile user stories and Given-When-Then criteria.
- **2. Domain Entity Extraction:**
 - Analyzes raw requirements documents to extract domain nouns (classes, attributes) and verbs (methods, associations).
- **3. Cross-Diagram Consistency Validation:**
 - Scans Use Case actors, Class diagrams, and Sequence lifelines to detect mismatched naming conventions and unmapped components.
- **4. Code-to-Model Reverse Engineering:**
 - Ingests legacy codebases (Java/TypeScript/Python) to auto-generate class hierarchies and dependency graphs for developer onboarding.

Human-in-the-Loop: Modeling Risks & Best Practices

- **Risks of Unchecked AI in System Modeling:**
 - **Hallucinated Associations:** Inventing fictitious inheritance or composition links that do not match business reality.
 - **Architectural Bloat:** Over-engineering class hierarchies with unnecessary design patterns instead of clean abstractions.
 - **Ghost Lifelines:** Inventing non-existent microservice endpoints in sequence diagrams.
- **The Golden Engineering Principle:**

AI Drafts the Diagram; The Human Architect Validates the Semantics!
Software architects must critically inspect generated models to ensure they reflect true domain boundaries and operational constraints.

”

Concept Check Question 6

What is the primary role of the software engineer/architect when using Generative AI for automated UML diagram generation?

- **A.** Manually coding every declarative diagram markup line without assistance.
- **B.** Validating domain semantics, structural constraints, and architectural simplicity.
- **C.** Eliminating traditional human code reviews and architecture design meetings.
- **D.** Automatically accepting all generated class relationships and microservice lifelines.



4.8 Conceptual Recap & References

"Models are the lingua franca of software engineering." „

Conceptual Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. The "Three Amigos" who unified UML at Rational Software were Grady Booch, Jim Rumbaugh, and .
2. diagrams delineate the operational boundary between an internal software system and external partner clouds.
3. In use case modeling, mandatory shared functionality is factored out using the relationship.
4. The architectural pattern that separates user interfaces, business logic, and persistent domain entities is .
5. In a UML class diagram, a filled black diamond () represents , where parts cannot exist without the whole.
6. In a UML class diagram, an open hollow diamond () represents , where parts maintain independent lifecycles.
7. A UML state machine transition label follows the formal syntax: Trigger Event [] / Action Effect.

References & Further Reading

- **Foundational Textbooks & Standards:**

- Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 5: System Modeling. Pearson.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
- Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.

- **Modern Declarative Diagramming & AI Tooling:**

- PlantUML Open-Source Standard: plantuml.com
- Mermaid.js JavaScript Diagramming Documentation: mermaid.js.org.